CAHIER DES CHARGES

Projet Okane Transfert

Plateforme de Gestion des Transferts d'Argent

exemple : Western Union / MoneyGram

0. Introduction et Contexte

Le projet OkaneTransfer s'inscrit dans la continuité des projets E-Banking 2.0 développé lors des promos précédente. Tandis que l'application E-Banking 2.0 couvrait la gestion de comptes bancaires classiques, TransferLink se focalise sur un segment complémentaire et à forte croissance : le transfert d'argent national et international entre particuliers, à la manière des opérateurs mondiaux tels que Western Union, MoneyGram.

Ce type de service touche des millions de personnes à travers le monde, en particulier les communautés de la diaspora qui envoient de l'argent à leurs familles dans les pays en développement. Les étudiants développeront une plateforme réaliste, sécurisée et modulaire permettant à une agence de transfert de gérer ses opérations du front-office jusqu'à la réconciliation comptable.

I. Objectifs du Projet

L'objectif principal est de concevoir et développer une application Web monolithique **(sans microservices)** de gestion des transferts d'argent. La solution doit répondre aux besoins de trois types d'acteurs principaux : l'administrateur central, les agents en agence, et les clients (expéditeurs/bénéficiaires).

1.1 Objectifs fonctionnels

- Permettre l'envoi et la réception de fonds entre particuliers, en national et à l'international
- Gérer un réseau d'agences et d'agents habilités à effectuer des opérations
- Assurer la traçabilité complète de chaque transaction via un code de retrait unique
- Intégrer la gestion multi-devises avec conversion en temps réel
- Produire des rapports financiers consolidés pour l'administration
- Offrir une interface client en ligne pour le suivi des envois

1.2 Objectifs techniques

- Développer une application monolithique robuste, maintenable et testée
- Implémenter une architecture en couches (Présentation / Service / Persistance)
- Sécuriser tous les accès via Spring Security avec JWT
- Fournir une API REST documentée (Swagger / OpenAPI 3)
- ~~Conteneuriser l'application avec Docker pour faciliter le déploiement~~
- ~~Mettre en place une intégration continue (CI/CD) avec GitHub Actions~~

II. Acteurs et Rôles

L'application distingue quatre profils utilisateurs, chacun disposant de droits stricts définis par Spring Security :

Profil	Rôle Spring	Responsabilités
Administrateur	`ROLE_ADMIN`	Configuration globale, gestion des devises, paramétrage des frais, supervision des agences, rapports consolidés, gestion des utilisateurs
Responsable d'agence	`ROLE_MANAGER`	Gestion des agents de son agence, validation des opérations sensibles, rapports d'agence, suivi des plafonds
Agent	`ROLE_AGENT`	Saisie des envois et des paiements, vérification d'identité des clients, remise de fonds contre code, gestion de la caisse
Client	`ROLE_CLIENT`	Consultation en ligne de l'historique de ses transferts, suivi en temps réel de ses envois, gestion de son profil

III. Description des Fonctionnalités

3.1 Espace Administration

Gestion des devises et des pays

- CRUD complet sur les devises (USD, EUR, MAD, GBP, etc.)
- Activation / désactivation d'un corridor de transfert (ex : Maroc → Sénégal)
- Mise à jour manuelle ou automatique (API externe) des taux de change
- Historique des variations de taux avec date et source

Paramétrage des frais et commissions

- Configuration des grilles tarifaires par tranche de montant
- Part agence vs part centrale paramétrable
- Simulation de frais avant validation
- Export des grilles tarifaires en PDF ou CSV

Gestion des agences

- Création et paramétrage des agences (nom, adresse, pays, plafonds journaliers)
- Affectation des responsables et des agents par agence
- Activation / suspension d'une agence
- Tableau de bord des performances par agence

Rapports et supervision

- Volume journalier / mensuel des transactions par corridor
- Chiffre d'affaires et commissions générées
- Alertes automatiques en cas d'anomalie (seuils dépassés, erreurs répétées)
- Journal d'audit complet (qui a fait quoi, quand)

3.2 Espace Agent — Front Office

Enregistrement d'un envoi

L'agent saisit les informations de l'expéditeur et du bénéficiaire, le montant, la devise et le pays de destination. Le système calcule automatiquement les frais selon la grille tarifaire active, affiche un récapitulatif clair, et génère un code de retrait unique (OTP à 8 chiffres) après validation du paiement.

- Formulaire d'expéditeur : nom, prénom, CIN/passeport, téléphone, pays
- Formulaire de bénéficiaire : nom, prénom, téléphone, pays de réception
- Calcul automatique des frais et du montant net reçu
- Génération du code de retrait unique (UUID court, alphanumérique)
- Impression ou envoi SMS/email du reçu à l'expéditeur

Paiement d'un transfert (retrait)

L'agent en agence de destination saisit le code de retrait communiqué au bénéficiaire. Le système vérifie la validité du code, le statut du transfert, et permet le paiement une fois l'identité du bénéficiaire contrôlée.

- Recherche par code de retrait ou numéro de téléphone du bénéficiaire
- Vérification du statut : EN_ATTENTE / PAYÉ / ANNULÉ / EXPIRÉ
- Saisie obligatoire du numéro de pièce d'identité du bénéficiaire
- Confirmation du paiement et mise à jour instantanée du statut
- Impression du reçu de paiement

Gestion de caisse

- Solde de caisse en temps réel
- Historique des opérations de la journée
- Clôture de caisse en fin de journée avec réconciliation
- Signalement d'un écart de caisse

3.3 Espace Client — Self-Service

Le client peut accéder à un espace personnel en ligne pour suivre ses transferts en cours, consulter son historique, et gérer son profil, sans dépendre d'un agent.

- Inscription et authentification sécurisée (email + mot de passe + 2FA SMS)
- Tableau de bord : transferts récents, statuts en temps réel
- Suivi détaillé d'un transfert par numéro de référence
- Historique complet avec filtres (date, montant, statut, corridor)
- Modification du profil et des préférences de notification
- Réception de notifications push ou email à chaque changement de statut

3.4 Fonctionnalités Innovantes

Pour se démarquer d'une simple application CRUD, les équipes doivent implémenter les fonctionnalités innovantes suivantes :

Module de conformité KYC/AML

- Vérification automatisée des pièces d'identité (reconnaissance de pattern)
- Contrôle des bénéficiaires contre une liste de surveillance (OFAC fictive)
- Génération automatique de déclarations de soupçon au-delà d'un seuil
- Tableau de bord de conformité pour l'administrateur

Assistant intelligent (Chatbot)

- Bot répondant aux questions fréquentes (statut, frais, délais)
- Intégration avec GPT ou Dialogflow via API
- Support multilingue (français, anglais, arabe)
- Escalade vers un agent humain si la question est hors périmètre

Transfert Mobile Money

- Envoi directement sur un compte Orange Money, Wave, M-Pesa (simulation)
- Notification par SMS lors de la réception
- Réconciliation automatique avec les relevés de l'opérateur
- Dashboard de suivi des transferts mobile money

IV. Spécifications Techniques

L'application doit être développée sous forme d'une application monolithique en couches, sans architecture microservices. Le découpage en packages doit respecter le principe de séparation des responsabilités.

4.1 Stack technologique obligatoire

Couche	Technologie	Détails
Backend	`Spring MVC 6 + Spring Security 6`	Architecture REST, JWT, sessions stateless
ORM / Données	`Spring Data JPA + Hibernate`	PostgreSQL ou MySQL (données métier)
API Documentation	`SpringDoc OpenAPI 3 (Swagger UI)`	Sans Spring Boot, configuration manuelle

Couche	Technologie	Détails
Frontend	Angular 17+ + Chart.js	SPA, routage, guards, intercepteurs HTTP
~~**Conteneurisation**~~	~~Docker + Docker Compose~~	~~Un conteneur par service (app, db, nginx)~~
~~**CI/CD**~~	~~GitHub Actions~~	~~Build, tests, déploiement automatique~~
~~**Tests**~~	~~JUnit 5 + Mockito + MockMvc~~	~~Unit, intégration, couverture > 70%~~
~~**Monitoring**~~	~~Spring Actuator + Micrometer~~	~~Métriques exposées, healthcheck~~

~~Ce n est pas obligatoire pour le moment.~~

4.2 Architecture applicative

L'application adopte une architecture en couches stricte, chaque couche ne communiquant qu'avec la couche immédiatement inférieure :

- Couche Présentation : contrôleurs REST annotés @RestController, DTOs d'entrée/sortie
- Couche Service : logique métier, gestion des transactions (@Transactional), validation
- Couche Persistance : repositories Spring Data JPA, entités JPA annotées
- Couche Infrastructure : configuration Spring Security, Swagger, CORS, gestion des exceptions

4.3 Sécurité

- Authentification stateless par JWT (access token 1h + refresh token 7j)
- Double authentification (2FA) par SMS OTP pour les opérations sensibles
- Hachage des mots de passe avec BCrypt (force 12)
- Protection CSRF désactivée (API stateless), CORS configuré
- Rate limiting sur les endpoints d'authentification (max 5 tentatives / 10 min)
- Journalisation de toutes les actions sensibles dans la table JournalAudit
- Chiffrement des numéros de pièces d'identité en base (AES-256)
- Validation stricte de toutes les entrées (Jakarta Validation + contraintes custom)

4.4 Exigences non fonctionnelles

- Performance : réponse < 500 ms pour 95% des requêtes sous charge normale
- ~~Disponibilité : l'application doit redémarrer automatiquement via Docker health checks~~
- Scalabilité horizontale : la session est stateless pour permettre le scale-out futur
- Conformité RGPD : pseudonymisation des données, droit à l'effacement implémenté
- Accessibilité : l'interface Angular doit respecter les critères WCAG 2.1 niveau A
- Internationalisation : prévoir l'ajout de nouvelles langues (i18n Angular)

V. Planification du Projet

Le projet est découpé en six phases correspondant aux jalons académiques du semestre. Chaque phase se termine par une livraison évaluée.

Phase	Intitulé	Durée	Livrables attendus
Phase 1	Spécifications	Sem. 1	Diagrammes UML (cas d'utilisation, classes, séquences), maquettes UI (Figma ou draw.io)
Phase 2	Conception	Sem. 2	Schéma de la base de données, architecture applicative, choix techniques justifiés
Phase 3	Développement	Sem. 3-4	Code source versionné sur GitHub, branches par fonctionnalité, Pull Requests documentées
Phase 4	Tests globaux & Démo	30 minutes	Soutenance juste après les examens

VI. Contraintes et Consignes Importantes

6.1 Contraintes de développement

- Application monolithique obligatoire — aucune architecture microservices
- Sans Spring Boot — configuration Spring manuelle (comme vu en cours)
- Le code source doit être hébergé sur un dépôt GitHub privé, partagé avec l'encadrant (moi)
- Commits réguliers obligatoires (au moins 3 commits par semaine par équipe)
- Aucune donnée réelle ne doit apparaître dans les jeux de tests — données fictives uniquement

6.2 Consignes de sécurité

- Ne jamais committer de secrets (mots de passe, clés API) en clair dans Git — utiliser des variables d'environnement
- Le .gitignore doit exclure application.properties contenant des secrets
- **Toute faille de sécurité identifiée lors de la soutenance entraîne une pénalité**

6.3 Livrables finaux

1. Dépôt GitHub avec README complet (installation, configuration, lancement)
2. ~~Rapport technique (conception, difficultés, solutions)~~
3. Diagrammes UML : cas d'utilisation, classes, séquences pour les 3 scénarios principaux
4. Application Swagger UI fonctionnelle documentant l'intégralité des endpoints
5. Présentation + démonstration live (pas de vidéos) +questions (30 minutes)